

PENGEMBANGAN APLIKASI MOBILE

State Hoisting, Perubahan Konfigurasi, dan Side-effect



Dosen Pengampu:

Muhammad Aminul Akbar, S.Kom., M. T

Disusun oleh:

Najwa Tazkiya Rahman - 245150401111042

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS BRAWIJAYA

2026

4.5.1 State Hoisting

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity:

```
fun CounterScreen(modifier: Modifier) {  
    var number by remember { mutableStateOf(0) }  
    val label by remember { mutableStateOf("&quot;Increment&quot;") }  
    Column(modifier) {  
        Text(text = "&quot;$number&quot;", fontSize = 72.sp)  
        Button(onClick = { number++ }) {  
            Text(text = "&quot;$label&quot;")  
        }  
    }  
}
```

Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

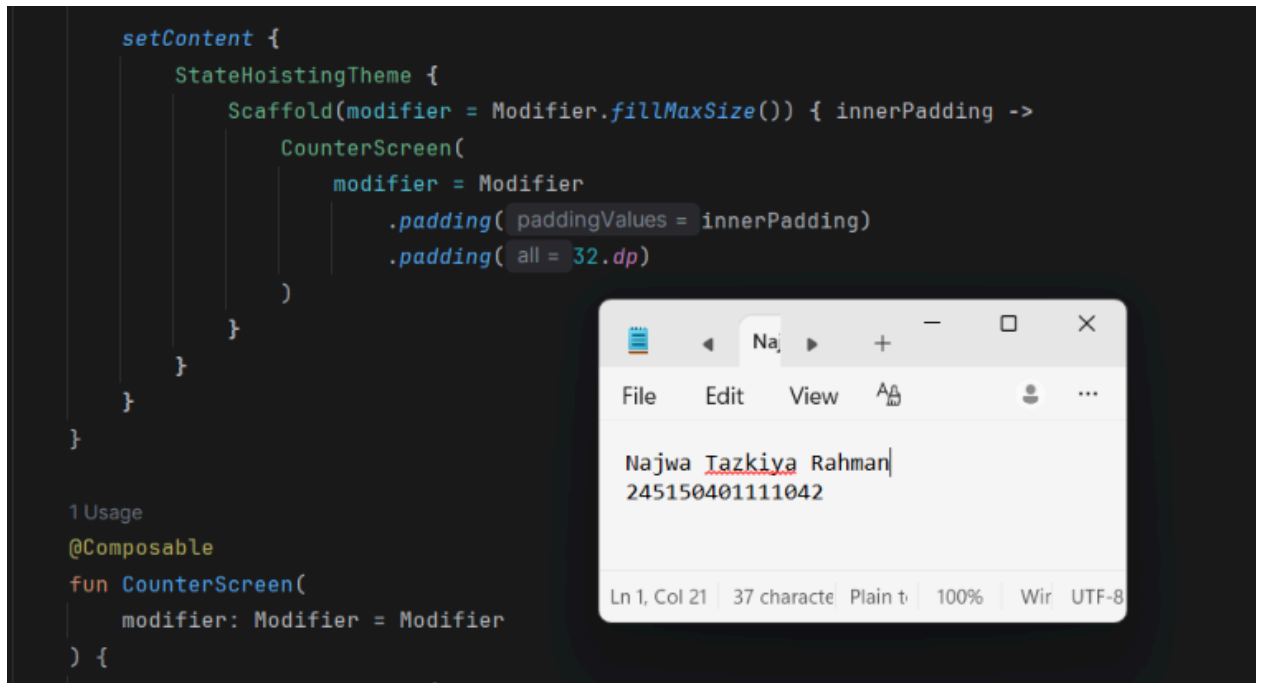
```
import androidx.compose.runtime.remember  
import androidx.compose.runtime.mutableStateOf  
import androidx.compose.runtime.getValue  
import androidx.compose.runtime.setValue  
import androidx.compose.material3.Button  
import androidx.compose.foundation.layout.Column  
import androidx.compose.ui.unit.dp  
import androidx.compose.ui.unit.sp
```

3. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah bagian kode program berikut di dalam fungsi onCreate():

```
Greeting(  
    name = &quot;Android&quot;,,  
    modifier = Modifier.padding(innerPadding)  
)
```

Kemudian ubah bagian tersebut menjadi:

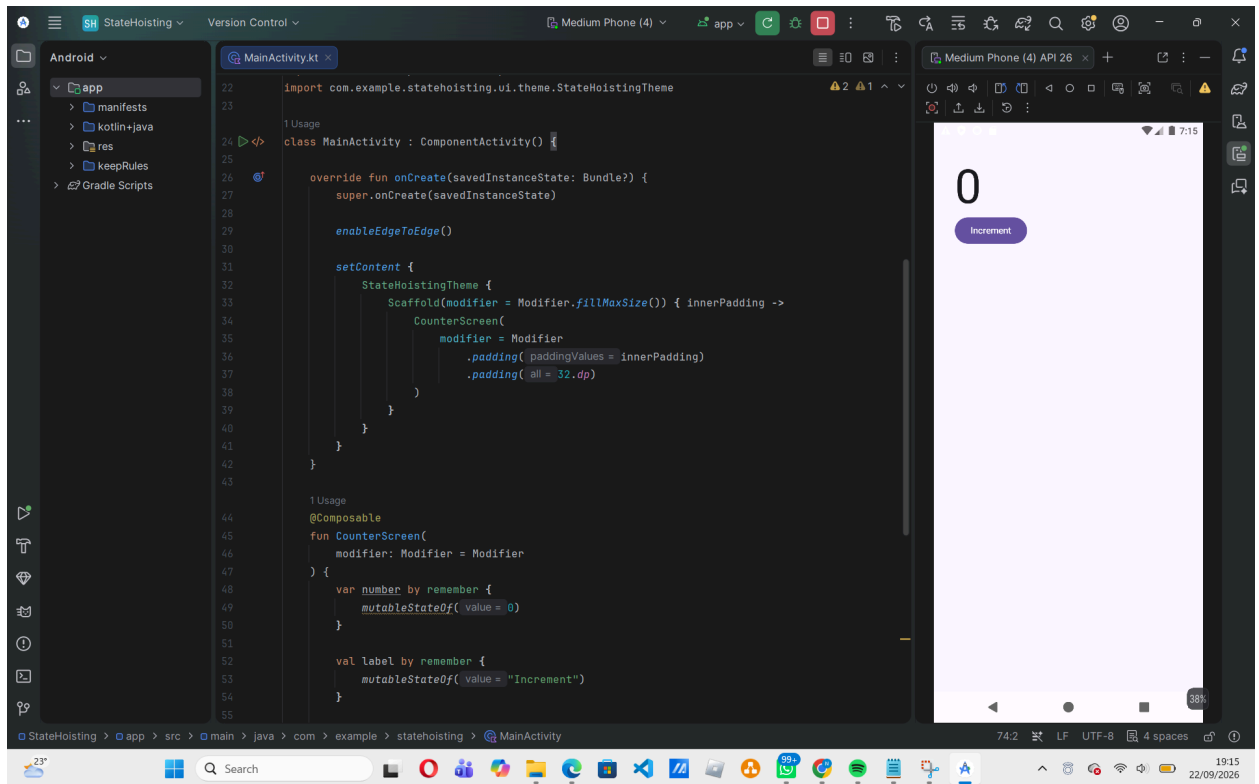
```
CounterScreen(modifier = Modifier.padding(32.dp))
```



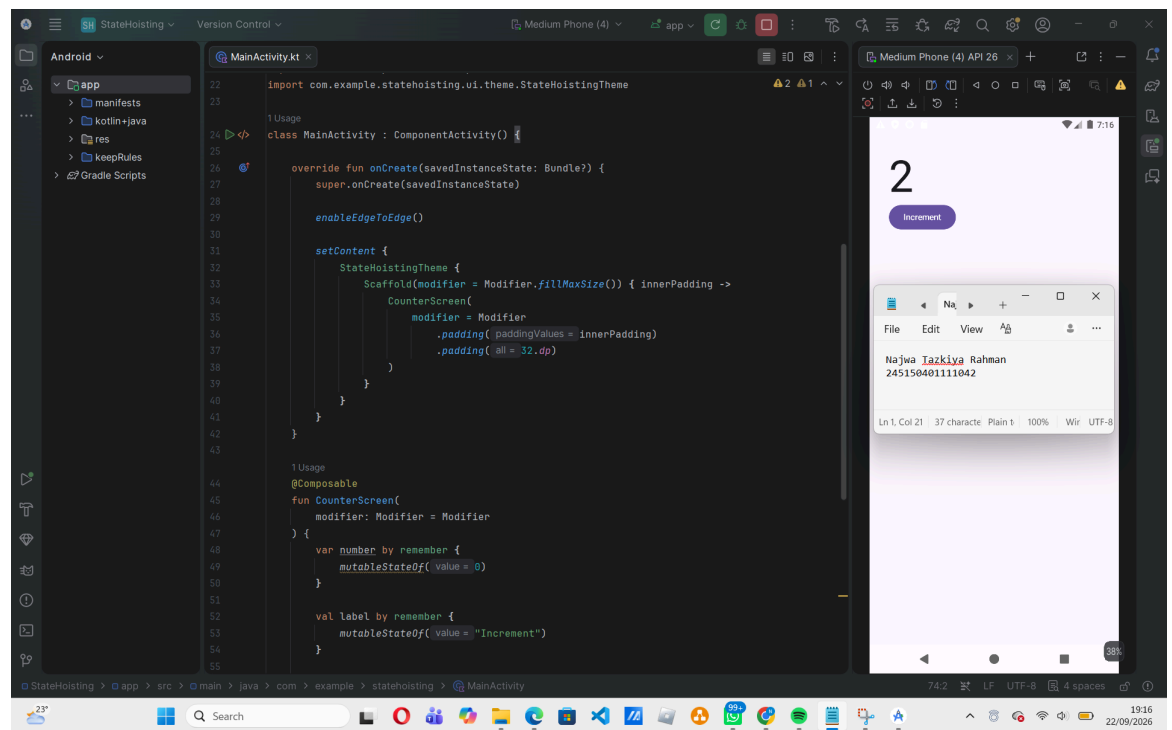
Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa Kali.



a. Apa yang terjadi setiap kali tombol “Increment” diklik?



Setiap kali tombol “Increment” diklik, nilai angka yang ditampilkan pada Text akan bertambah 1. Misalnya dari 0 menjadi 1, kemudian 2 seperti contoh gambar diatas.

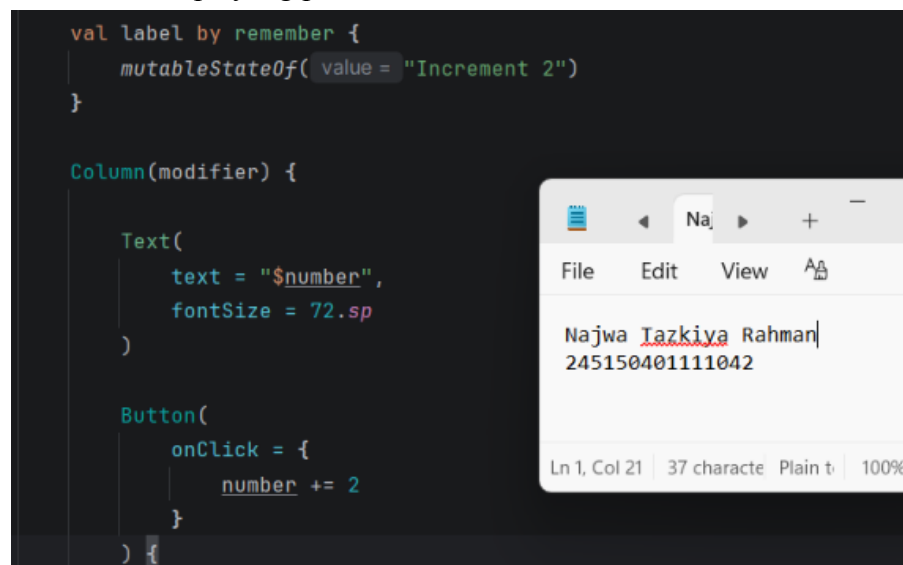
b. Apa yang menyebabkan hal tersebut terjadi? Hal tersebut terjadi karena terdapat event `onClick` pada tombol yang menjalankan perintah `number++` yang mana perintah tersebut menambahkan nilai variabel `number` sebesar 1 setiap kali tombol ditekan.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut? Variabel yang berperan adalah `number`. Variabel tersebut digunakan pada `Text(text = "$number", fontSize = 72.sp)` yang mana nilai `number` menentukan angka yang ditampilkan pada Text.

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan “Increment” tersebut? Variabel yang digunakan adalah `label`. Variabel tersebut didefinisikan dengan `val label by remember { mutableStateOf("Increment") }`, kemudian digunakan pada `Text(text = "$label")` sehingga label tombol yang ditampilkan adalah Increment.

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable `CounterScreen` agar setiap kali tombol “Increment” diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol “Increment” menjadi “Increment 2”.

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?



```
val label by remember {  
    mutableStateOf( value = "Increment 2")  
}  
  
Column(modifier) {  
  
    Text(  
        text = "$number",  
        fontSize = 72.sp  
    )  
  
    Button(  
        onClick = {  
            number += 2  
        }  
    )  
}
```

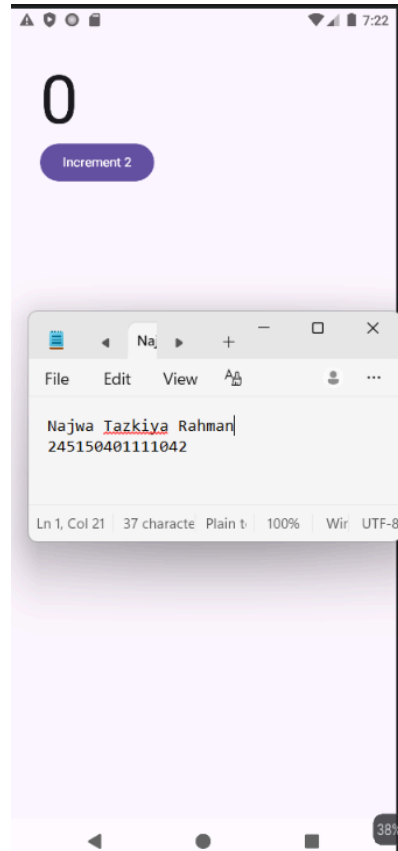
The screenshot shows a code editor with the above Kotlin code. To the right, a preview window displays a text input field containing the name "Najwa Iazkiya Rahman" and the ID "2451504011111042". The status bar at the bottom of the preview window indicates "Ln 1, Col 21 | 37 character | Plain text | 100%".

Perubahan dilakukan pada fungsi `CounterScreen()`, khususnya pada dua bagian. Yang pertama mengubah `number++` menjadi `number += 2`. Kedua mengubah

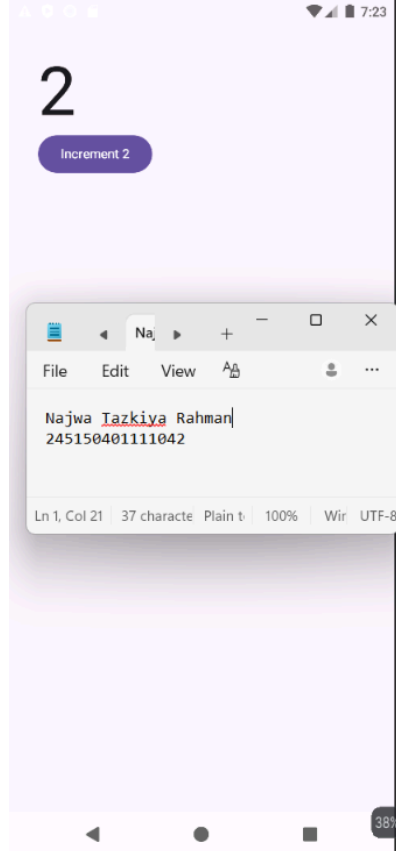
label “Increment” menjadi “Increment 2”. Dan kedua perubahan ini dilakukan pada kode CounterScreen sebelum proses state hoisting.

b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Sebelum



Sesudah



3. Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen. Ubah signature kode program pada bagian fungsi Composable CounterScreen dari yang sebelumnya:

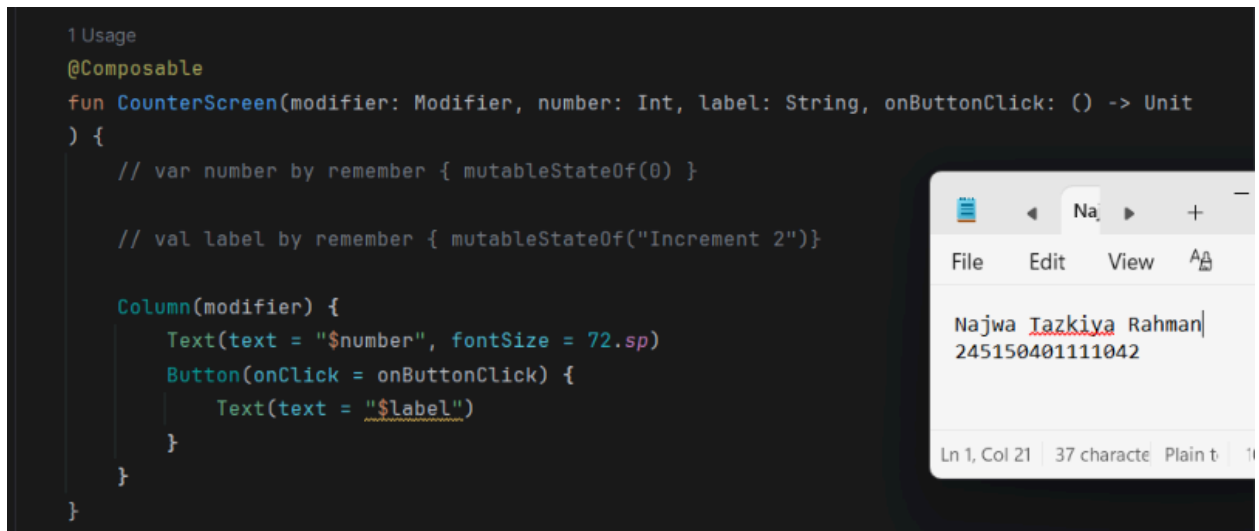
```
fun CounterScreen(modifier: Modifier)
```

Menjadi:

```
fun CounterScreen(modifier: Modifier,  
    number: Int, label: String, onClick: () -> Unit)
```

Ubah body di dalam Composable CounterScreen menjadi:

```
// var number by remember { mutableStateOf(0) }
// val label by remember { mutableStateOf(""Increment"") }
Column(modifier) {
    Text(text = &quot;$number&quot;, fontSize = 72.sp)
    Button(onClick = onButtonClick) {
        Text(text = &quot;$label&quot;,)
    }
}
```



Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

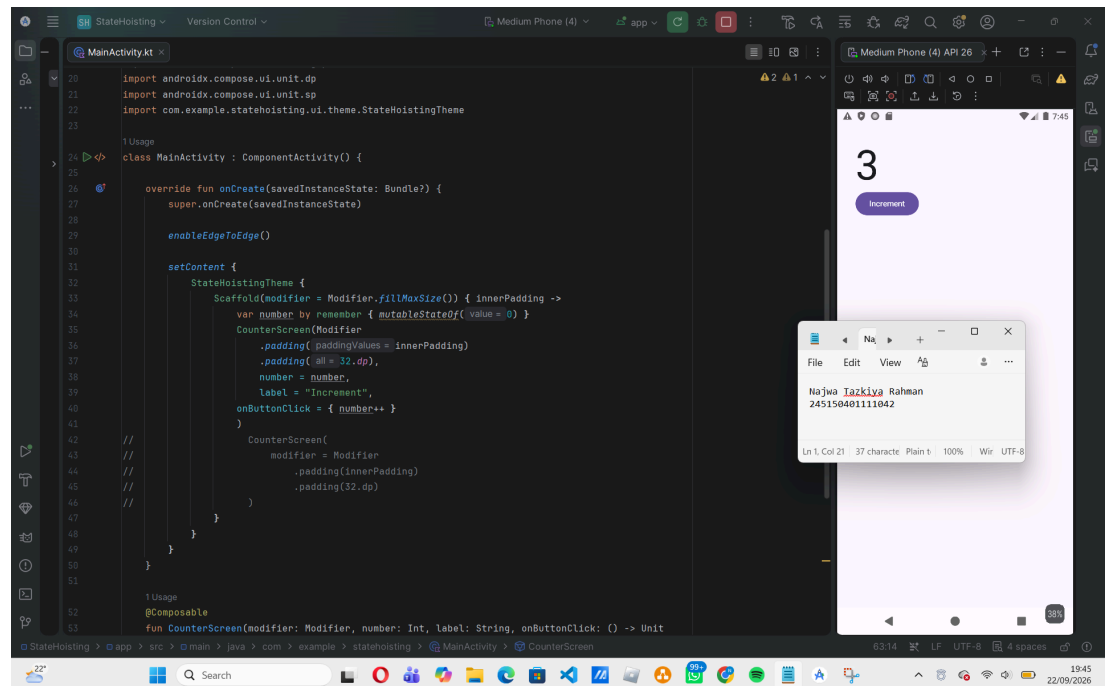
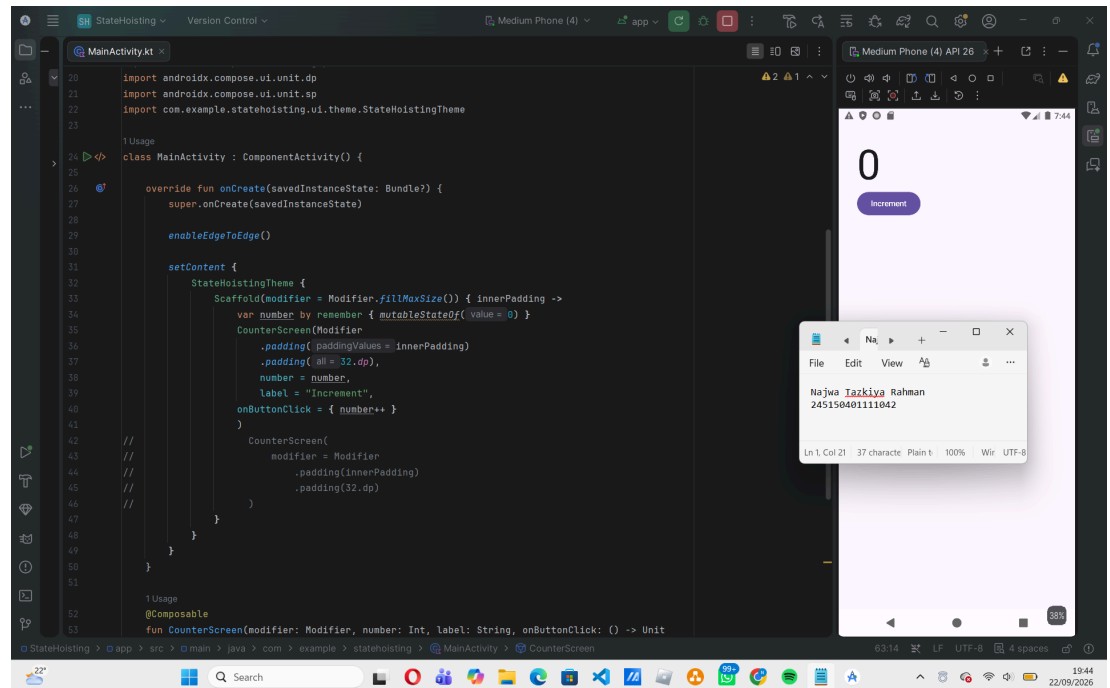
Menjadi:

```
var number by remember { mutableStateOf(0) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = &quot;Increment&quot;,
    onButtonClick = { number++ }
)
```

```
setContent {
    StateHoistingTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            var number by remember { mutableStateOf( value = 0) }
            CounterScreen(Modifier
                .padding( paddingValues = innerPadding)
                .padding( all = 32.dp),
                number = number,
                label = "Increment",
                onClick = { number++ }
            )
            CounterScreen(
                modifier = Modifier
                    .padding(innerPadding)
                    .padding(32.dp)
            )
        }
    }
}
```

Perhatikan bahwa deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?



Iya, program tetap berjalan dengan baik dan menghasilkan tampilan dan fungsi yang sama seperti tahap sebelumnya. Perbedaannya adalah lokasi pengelolaan state. Sebelumnya state number berada di dalam CounterScreen(), sedangkan setelah state hoisting, state tersebut berada di onCreate() dan dikirimkan sebagai parameter ke CounterScreen().

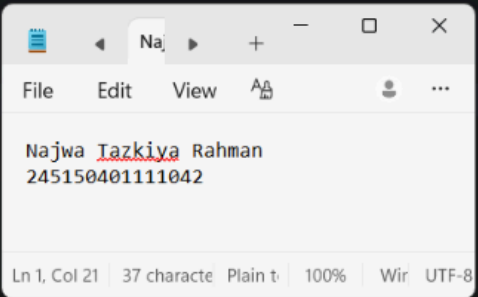
b. Di bagian kode program manakah saat ini proses increment dilakukan? Proses increment sekarang dilakukan pada lambda `onButtonClick` ketika `CounterScreen()` dipanggil. Jadi, proses perubahan nilai number berada di dalam `onCreate()`, bukan lagi di dalam `CounterScreen()`.

c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button “Increment” dideklarasikan? State number dideklarasikan di dalam `onCreate()`, sedangkan Text untuk menampilkan angka dan button tetap dideklarasikan di dalam `CounterScreen()`. Dengan itu, `CounterScreen()` bertugas menampilkan UI, sedangkan state dan event dikelola dari luar Composable.

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

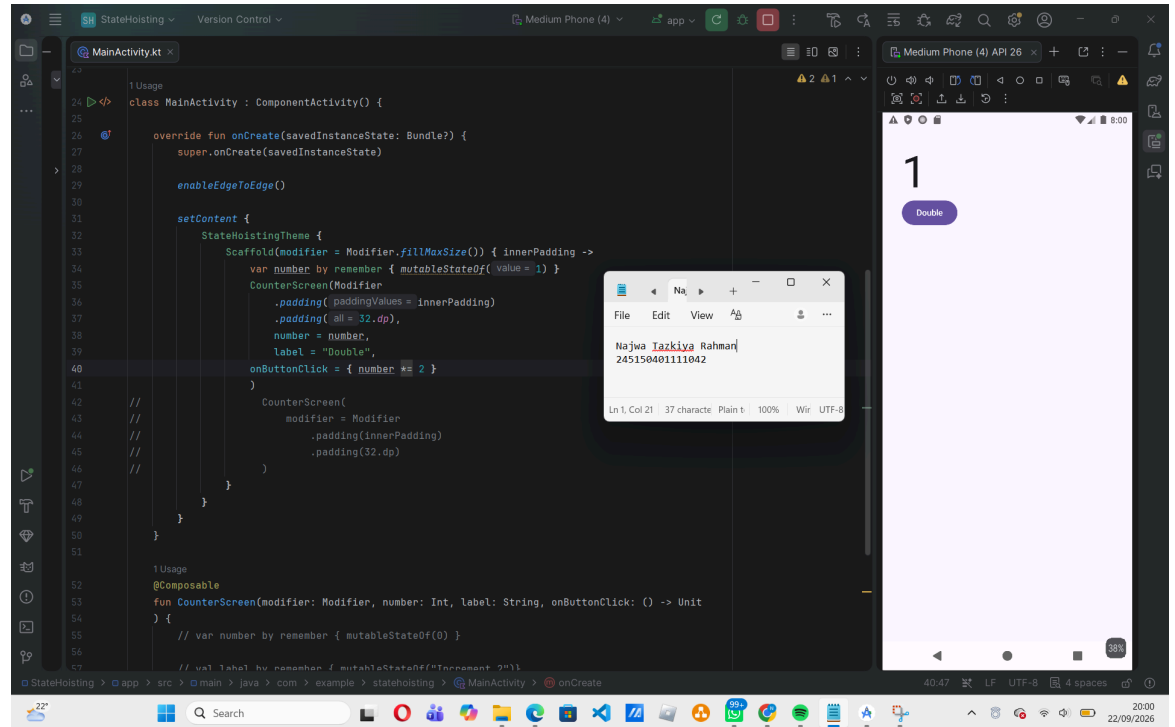
- Label tombol “Increment” menjadi “Double”
- Nilai awal variabel number dari 0 menjadi 1.
- Ubah fungsi/lambda `onButtonClick` pada pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
- Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
setContent {
    StateHoistingTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            var number by remember { mutableStateOf( value = 1) }
            CounterScreen(Modifier
                .padding( paddingValues = innerPadding)
                .padding( all = 32.dp),
                number = number,
                label = "Double",
                onButtonClick = { number *= 2 }
            )
            CounterScreen(
                modifier = Modifier
                    .padding(innerPadding)
                    .padding(32.dp)
            )
        }
    }
}
```

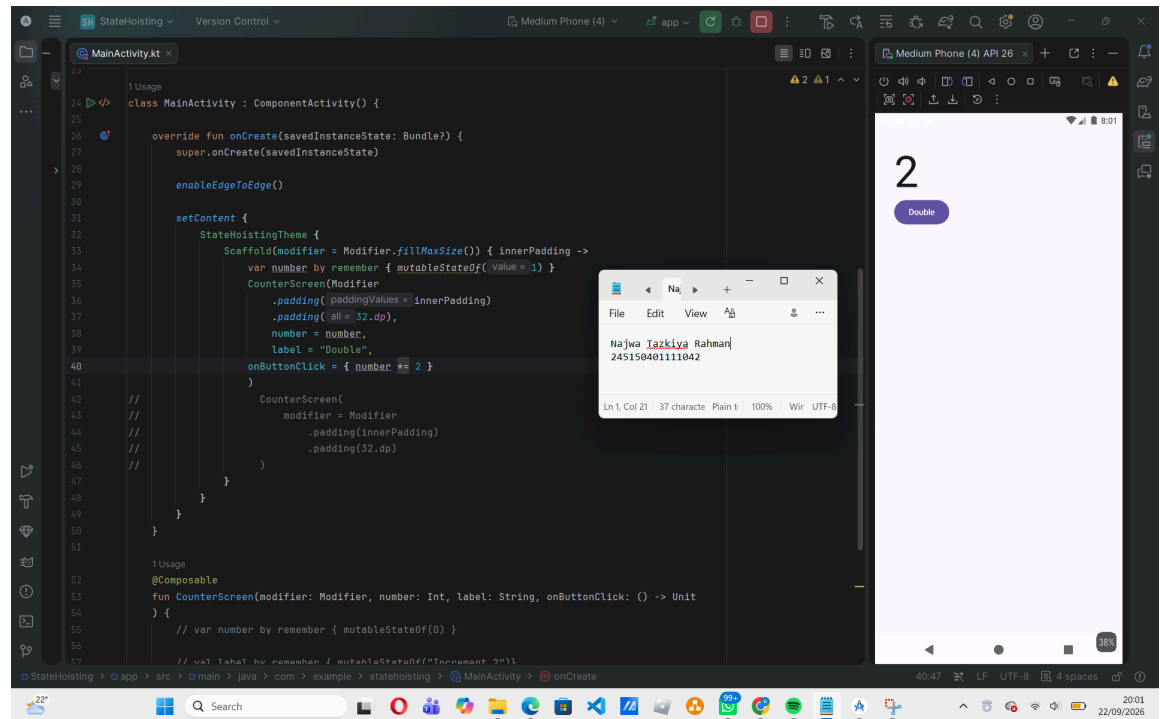


6. Jalankan program,
- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Sebelum



Sesudah



b. Di bagian manakah dari kode program pada percobaan tahap nomor 4 dan tahap nomor 5 dilakukan? Perubahan dilakukan pada fungsi onCreate(), terutama pada deklarasi state number, label Double, dan lambda onClick.

c. Apakah ada perubahan yang dilakukan dari tahap nomor 4 dan tahap nomor 5 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada perubahan logika utama pada CounterScreen(). CounterScreen() tetap menerima number, label, dan onClick sehingga menampilkan UI berdasarkan parameter tersebut. Perubahan logika penggantian dilakukan di onCreate().

7. Apa arti dari jawaban pada tahap nomor 6 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

State hoisting membuat state dan logika aplikasi dipisahkan dari tampilan UI. Dengan demikian, ketika kebutuhan fungsional berubah, misalnya dari tombol Increment menjadi Double, perubahan dapat dilakukan pada bagian pengelolaan state dan event tanpa harus mengubah struktur utama CounterScreen(). Hal ini membuat kode menjadi lebih fleksibel, mudah dikelola, dan lebih mudah digunakan kembali ketika terdapat perubahan kebutuhan pada antarmuka pengguna.

Tugas Praktik

Lembar Kerja Praktik

4.5.2 Perubahan Konfigurasi

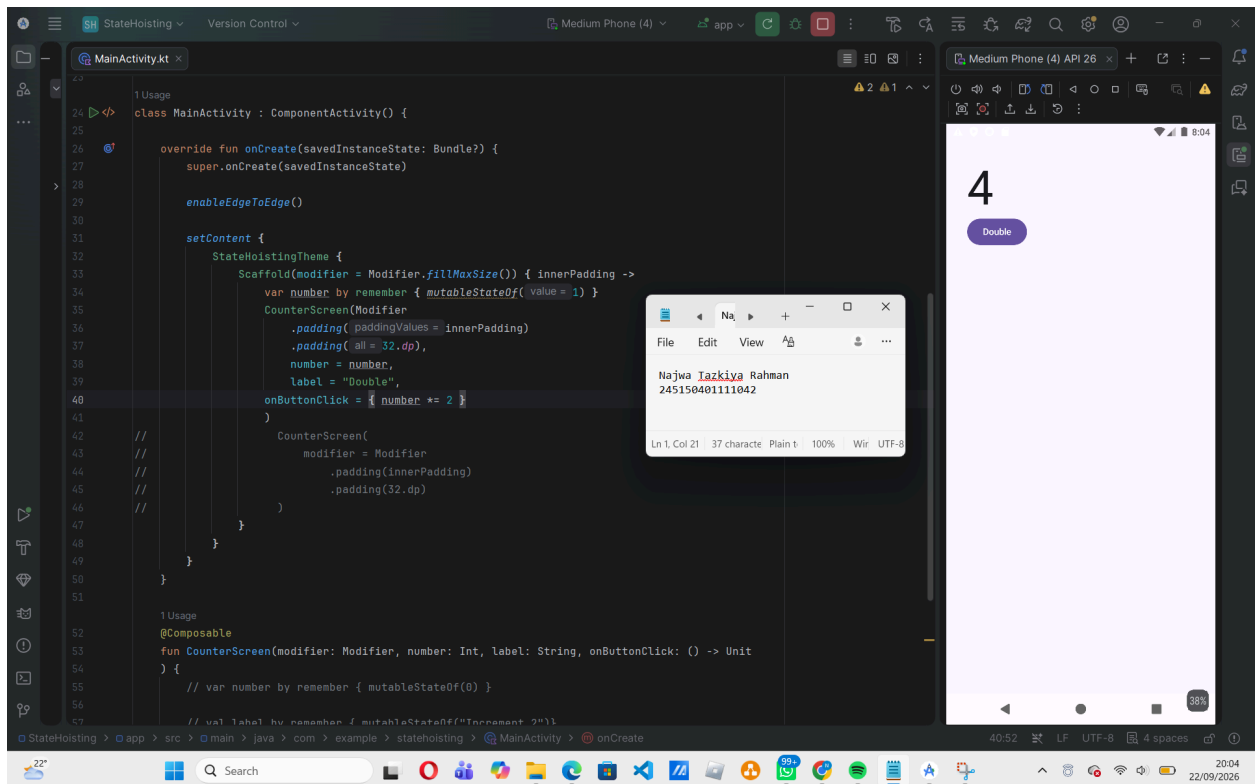
Pra-Percobaan

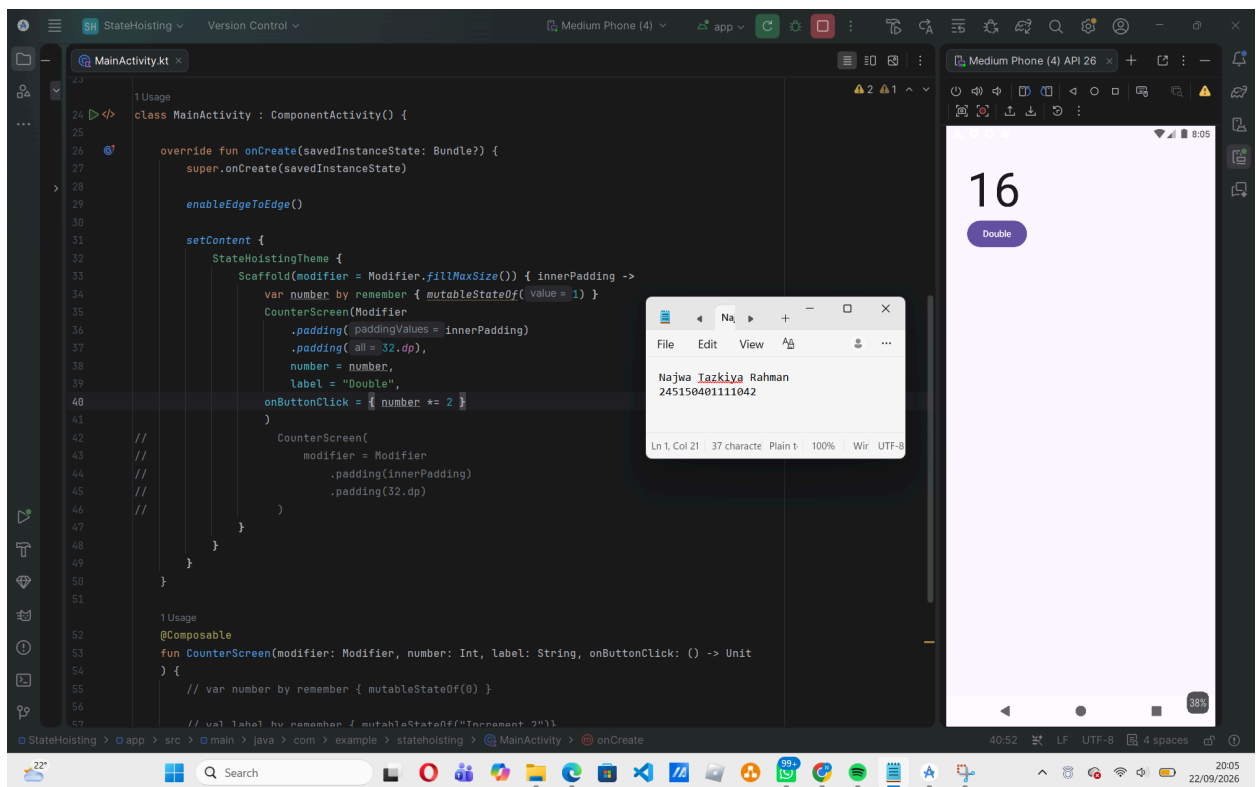
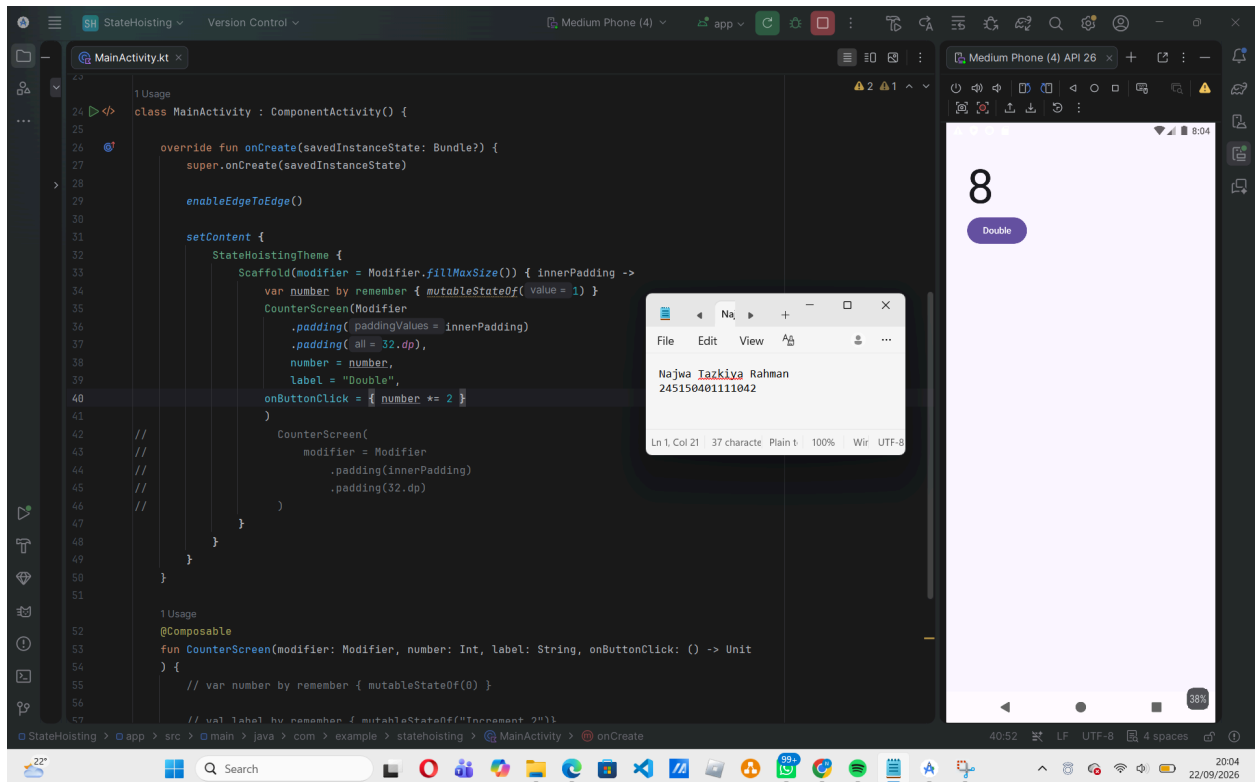
1. Untuk percobaan terkait Perubahan Konfigurasi aplikasi Android, gunakan Project Android Studio yang sama dengan kode program percobaan State Hoisting setelah menyelesaikan tahap nomor 6 percobaan tersebut.

2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel number).

Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol “Double” sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol “Double” diklik beberapa kali, ambil screenshot tampilan antarmukanya.

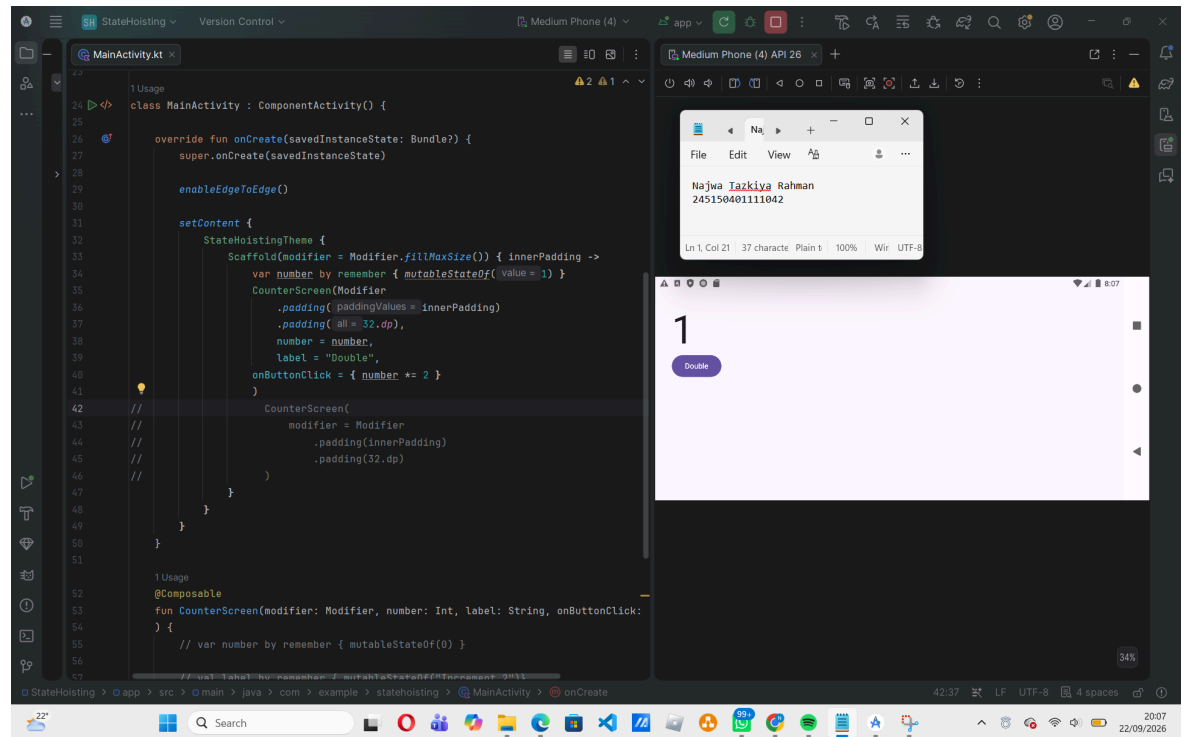




2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh

Android.

- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



Ketika tampilan berubah menjadi landscape, angka yang ditampilkan juga ikut berubah menjadi angka 1.

- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Pada percobaan awal, jika nilai number sudah berubah kemudian perangkat diputar dari portrait ke landscape, nilai dapat kembali ke 1, yaitu nilai awal. Hal tersebut terjadi karena perubahan konfigurasi seperti perubahan orientasi dapat menyebabkan Activity dibuat ulang. State yang hanya menggunakan remember tidak dipertahankan setelah proses tersebut.

Ubah bagian kode program di dalam fungsi onCreate() seperti berikut ini:

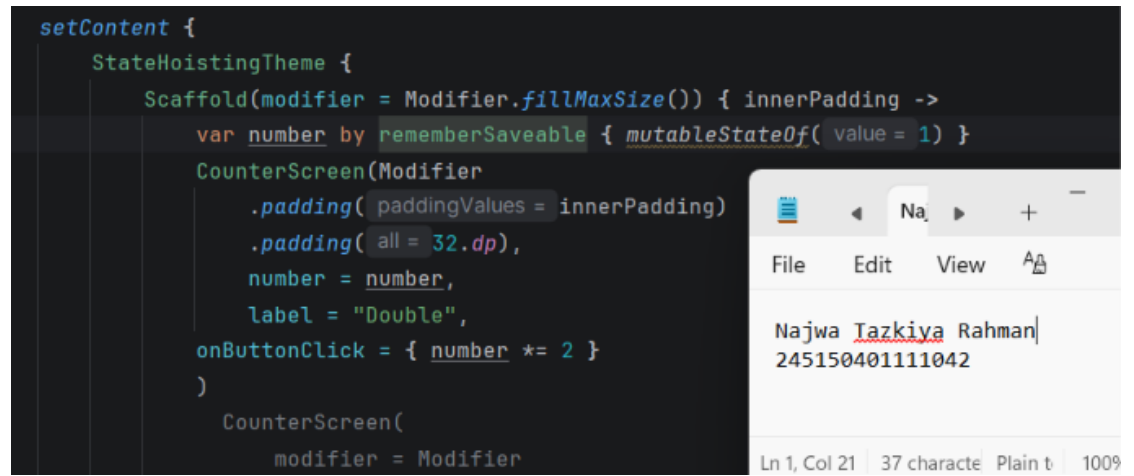
```
var number by remember { mutableStateOf(1) }
```

Menjadi:

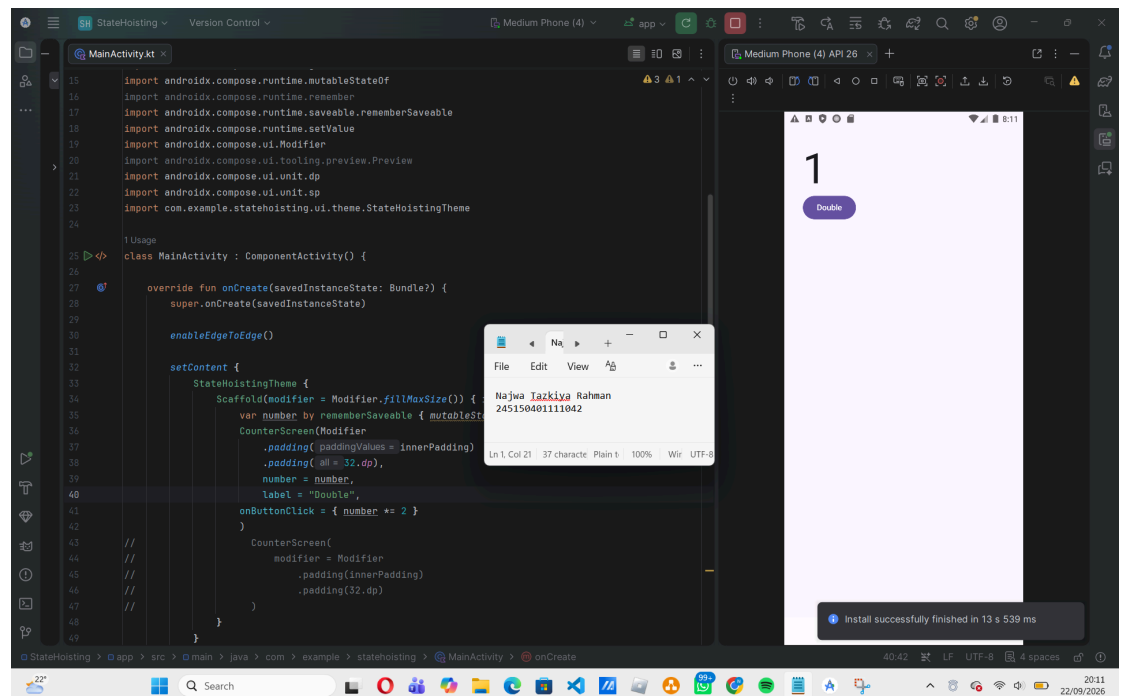
```
var number by rememberSaveable { mutableStateOf(1) }
```

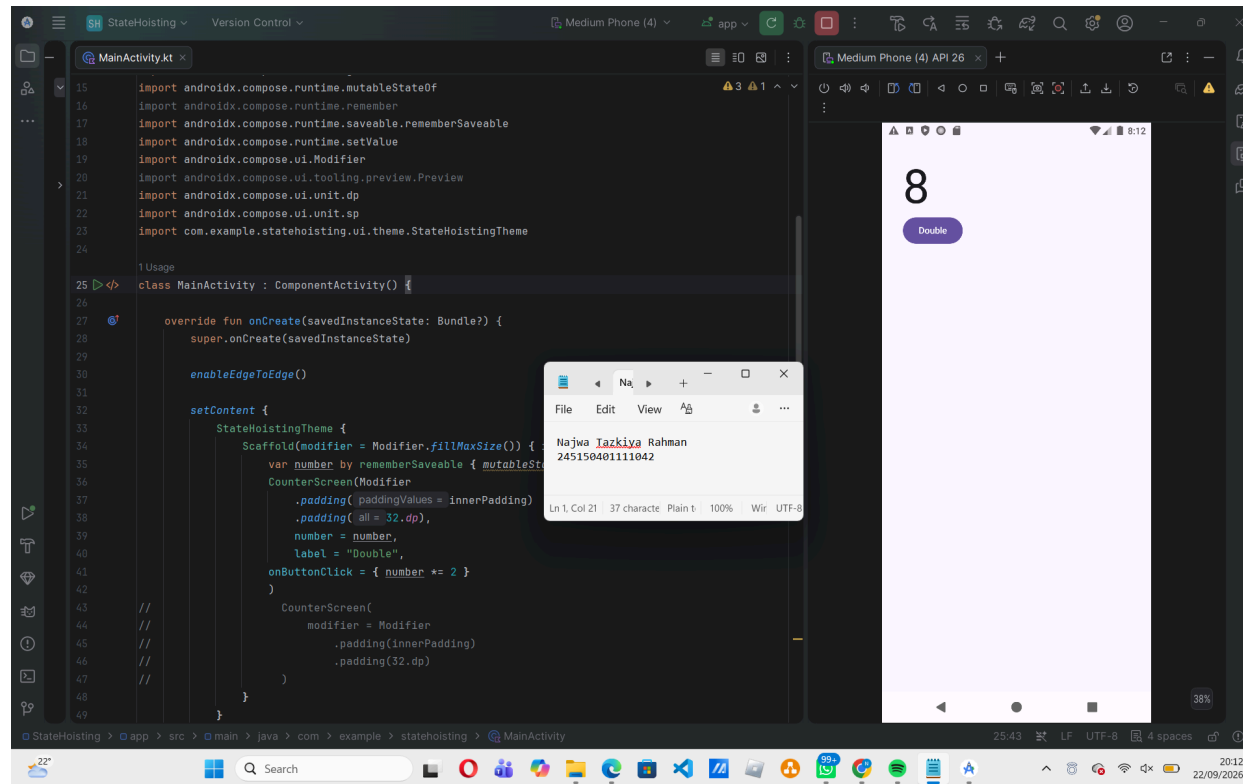
Lakukan import dependency atas perubahan konstruk remember menjadi rememberSaveable yang dilakukan (dengan memposisikan kursor pada konstruk

rememberSaveable tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela editor Android Studio).



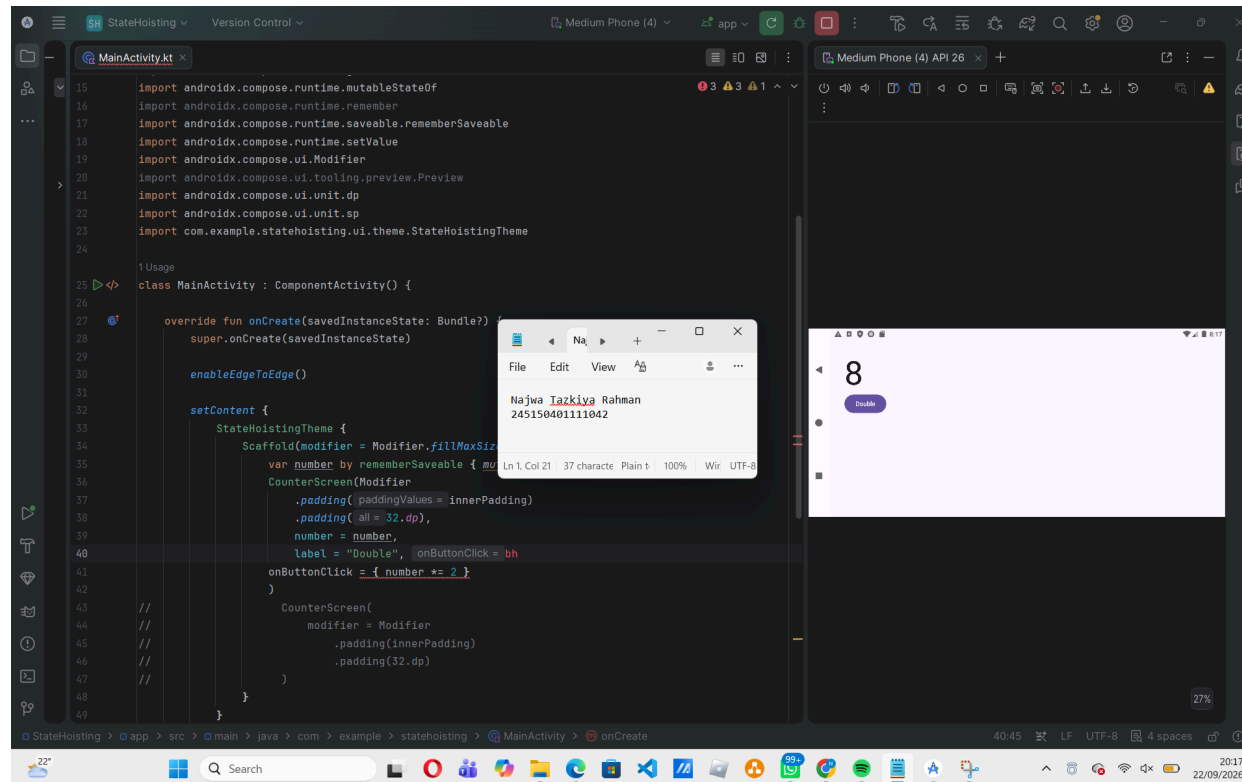
Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol “Double” sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol “Double” diklik beberapa kali, ambil screenshot tampilan antarmukanya.





3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



Angka yang ditampilkan saat landscape tetap sama dengan angka yang ditampilkan ketika portrait sebelumnya.

b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan adalah tetap 8, karena setelah kode diubah menjadi `var number by rememberSaveable { mutableStateOf(1) }` nilai number akan tetap mempertahankan nilai terakhir setelah perangkat diputar dari portrait ke landscape, jadi nilai tidak kembali ke 1.

c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

`remember` digunakan untuk mempertahankan state selama recomposition, tetapi state tersebut tidak dirancang untuk dipertahankan ketika Activity mengalami perubahan konfigurasi. Sedangkan `rememberSaveable` digunakan untuk menyimpan state yang dapat dipulihkan kembali setelah perubahan konfigurasi, seperti rotasi layar.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh kasus nyata adalah form pendaftaran akun. Misalnya pengguna sudah mengisi nama, email, nomor telepon, dan alamat. Kemudian pengguna memutar perangkat dari

portrait ke landscape. Dengan menggunakan `rememberSaveable`, data yang sudah dimasukkan dapat dipertahankan sehingga pengguna tidak perlu mengisi ulang dari awal. Manfaatnya adalah meningkatkan kenyamanan pengguna dan mencegah data input hilang ketika terjadi perubahan konfigurasi.

Tugas Praktik
Lembar Kerja Praktik

4.5.3 LaunchedEffect

Pra-Percobaan 1

1. Untuk percobaan ini, gunakan Project Android Studio yang sama dengan kode program percobaan Perubahan Konfigurasi setelah menyelesaikan tahap nomor 4 percobaan tersebut.

2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel `number`).

3. Mengaitkan composable `SnackBar` di dalam `Scaffold`.

Di dalam fungsi `onCreate()` dan di dalam lambda `Theme{}` sebelum pemanggilan composable `Scaffold` dilakukan, tambahkan deklarasi `host state` untuk menyimpan state composable `SnackBar` yang akan digunakan untuk menampilkan pesan:

```
val snackbarHostState = remember { SnackbarHostState() }
```

Kemudian ubah bagian pemanggilan fungsi Composable `Scaffold` di dalam fungsi `onCreate()` yang sebelumnya:

```
Scaffold(modifier = Modifier.fillMaxSize())
```

menjadi:

```
Scaffold(  
    modifier = Modifier.fillMaxSize(),  
    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }  
)
```

4. Menerapkan fitur Side-effect dengan `LaunchedEffect`. Dengan posisi kursor tetap berada di dalam lambda `Scaffold`, setelah pemanggilan composable

CounterScreen tambahkan kode program side-effect berikut ini:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar("Number $number is shown!")  
}
```

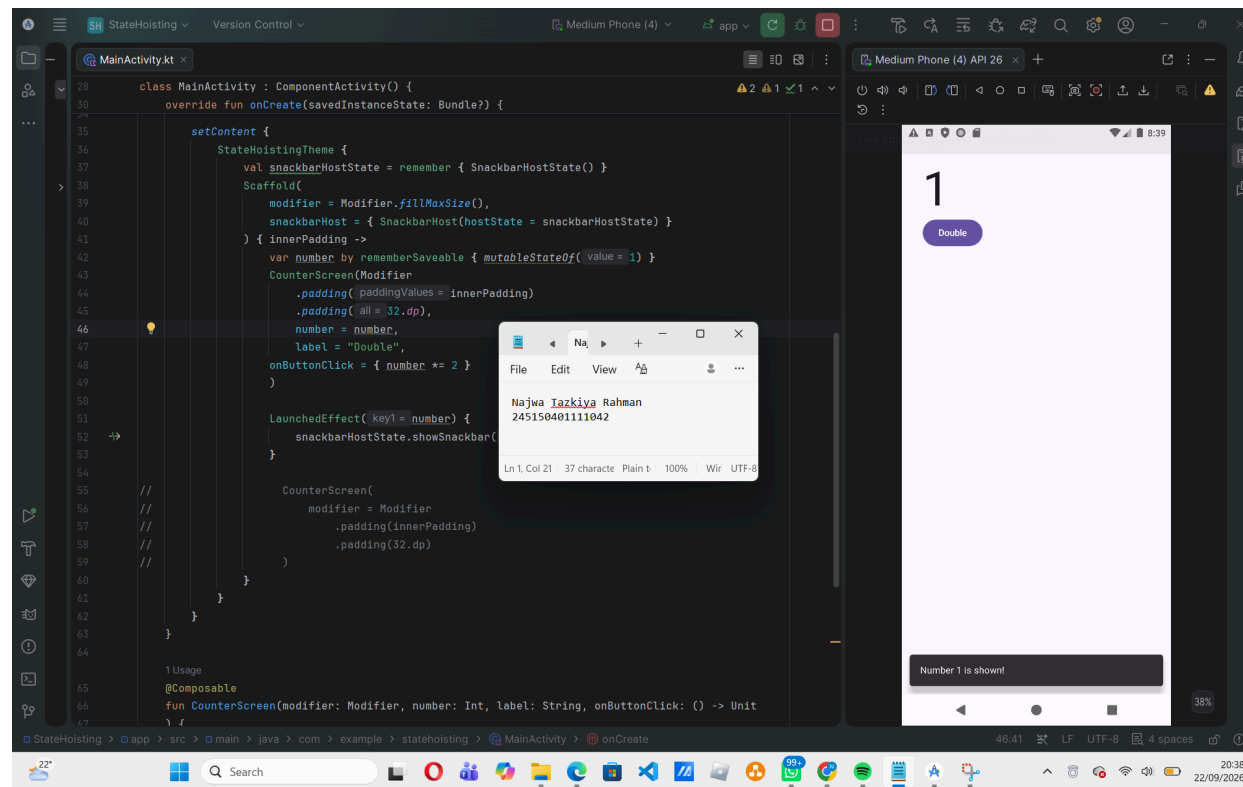
5. Lakukan import semua dependency yang diperlukan dari perubahan kode program yang telah dilakukan (dengan melakukan refactoring melalui Alt-Enter pada bagian yang ditandai merah di dalam jendela editor Android Studio) agar perubahan kode program yang dilakukan dapat di-compile dan dijalankan dengan baik.

Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composable CounterScreen().

Aktivitas Percobaan

1. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik button “Double” apa yang ditampilkan di dalam layar?



Ketika aplikasi pertama kali dijalankan, akan muncul SnackBar dengan pesan “Number 1 is shown!”. Pesan tersebut menampilkan nilai number saat aplikasi pertama kali dijalankan.

b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

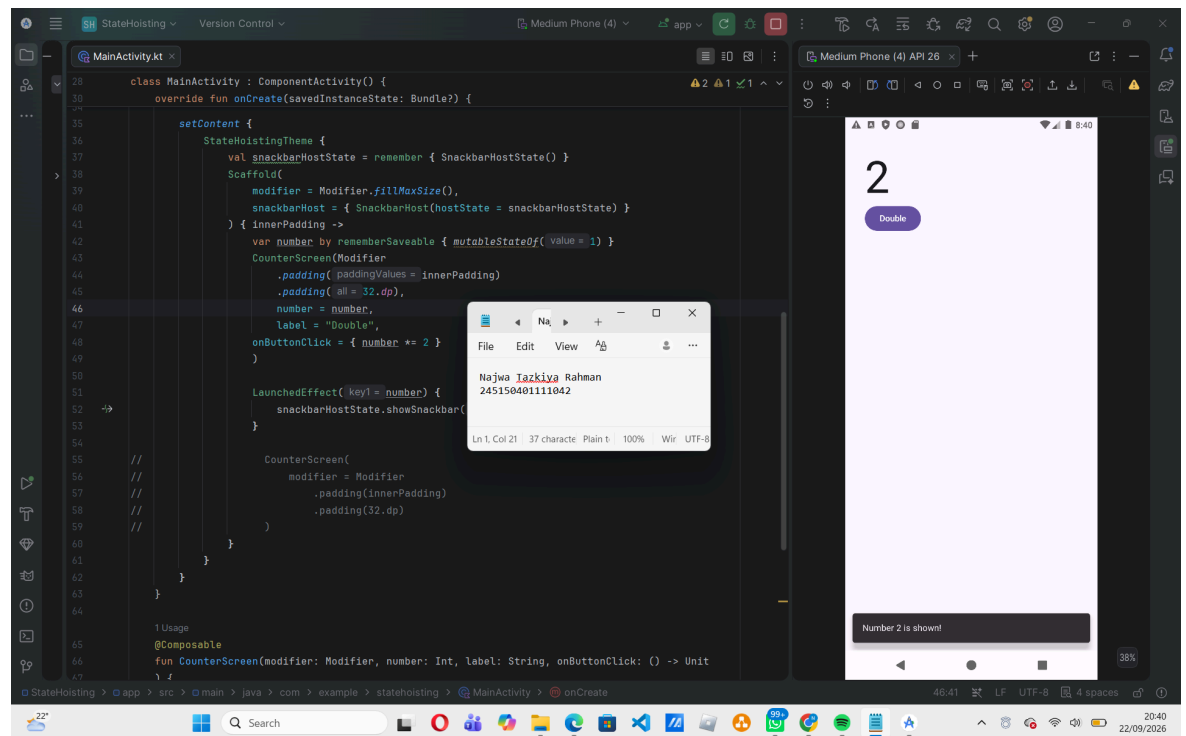
Karena `LaunchedEffect(number)` dijalankan ketika Composable pertama kali masuk ke composition. Pada saat itu number memiliki nilai awal sehingga efek tersebut langsung menjalankan perintah untuk menampilkan Snackbar. Jadi, pengguna tidak perlu menekan tombol terlebih dahulu.

c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Snackbar dipicu oleh perubahan atau penggunaan nilai number sebagai key pada `LaunchedEffect(number)`. Kemudian perintah untuk menampilkan Snackbar terdapat di kode `snackbarHostState.showSnackbar("Number $number is shown!")`

2. Klik button “Double” satu kali,

a. Apa yang terjadi?



Ketika tombol Double diklik, nilai number menjadi dua kali lipat. Setelah nilai berubah, `LaunchedEffect(number)` akan dijalankan kembali dan Snackbar menampilkan nilai terbaru.

b. Mengapa hal tersebut terjadi? Jelaskan!

Karena number digunakan sebagai key pada `LaunchedEffect`. Ketika tombol Double ditekan, nilai number berubah. Perubahan key tersebut menyebabkan

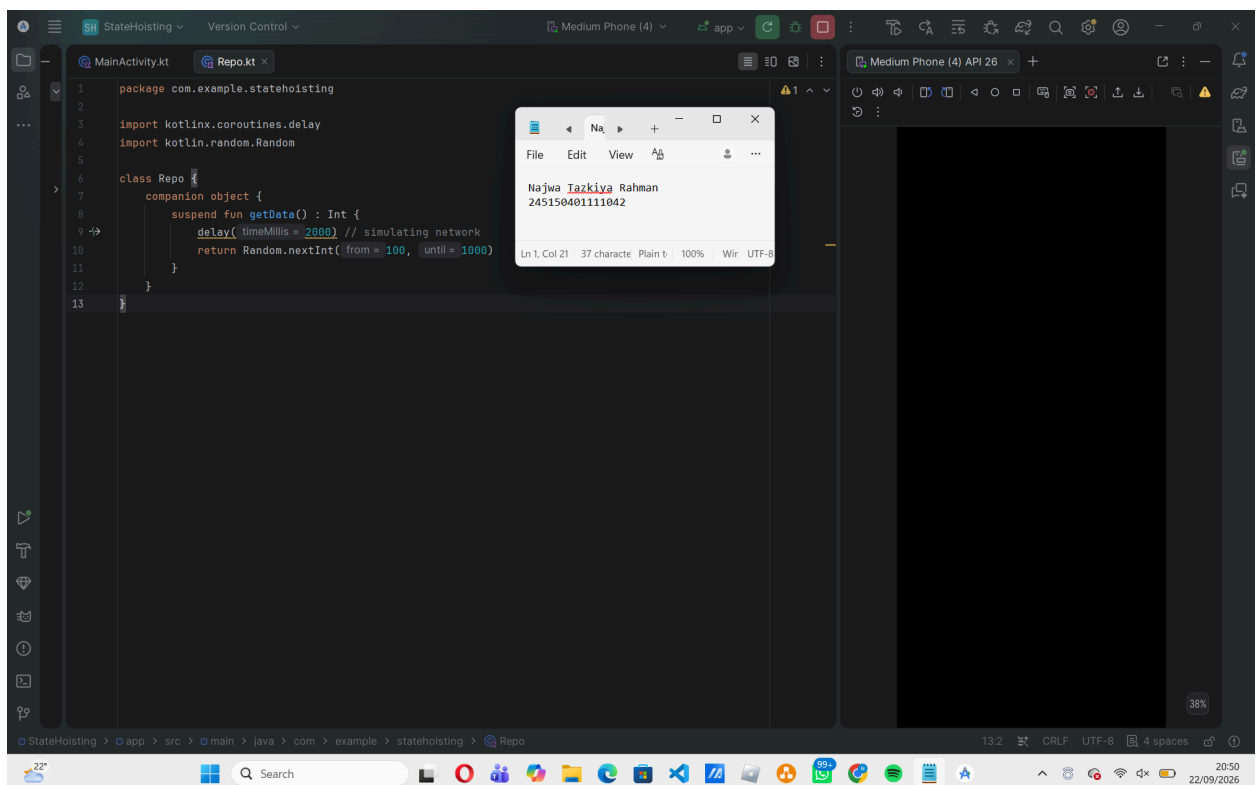
efek dijalankan kembali sehingga SnackBar menampilkan nilai number yang baru.

Pra-Percobaan 2

1. Untuk percobaan ini, lanjutkan Project Android Studio yang sama dengan kode program percobaan Side-effect sebelumnya.
2. Di dalam package yang sama dengan class MainActivity, buat sebuah class Kotlin baru bernama Repo (file: Repo.kt) dengan struktur class dan fungsi static getData() seperti pada kode program berikut ini:

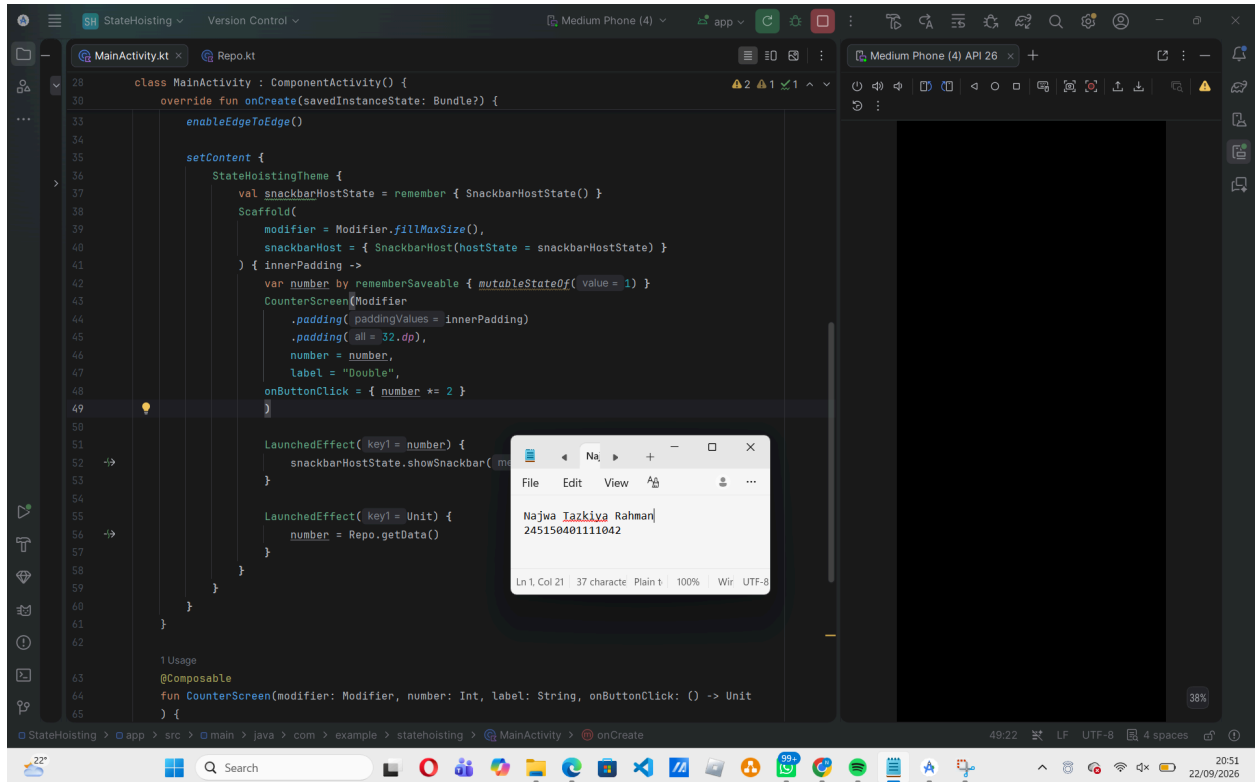
```
class Repo {  
    companion object {  
        suspend fun getData() : Int {  
            delay(2000) // simulating network  
            return Random.nextInt(100, 1000)  
        }  
    }  
}
```

Lakukan import dependency untuk fungsi delay() dan class Random. Fungsi delay(2000) ini digunakan untuk menyimulasikan waktu yang dibutuhkan untuk mengambil data melalui koneksi internet.



3. Di dalam fungsi onCreate(), setelah LaunchedEffect pada percobaan sebelumnya, tambahkan deklarasi fungsi LaunchedEffect baru berikut ini:

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

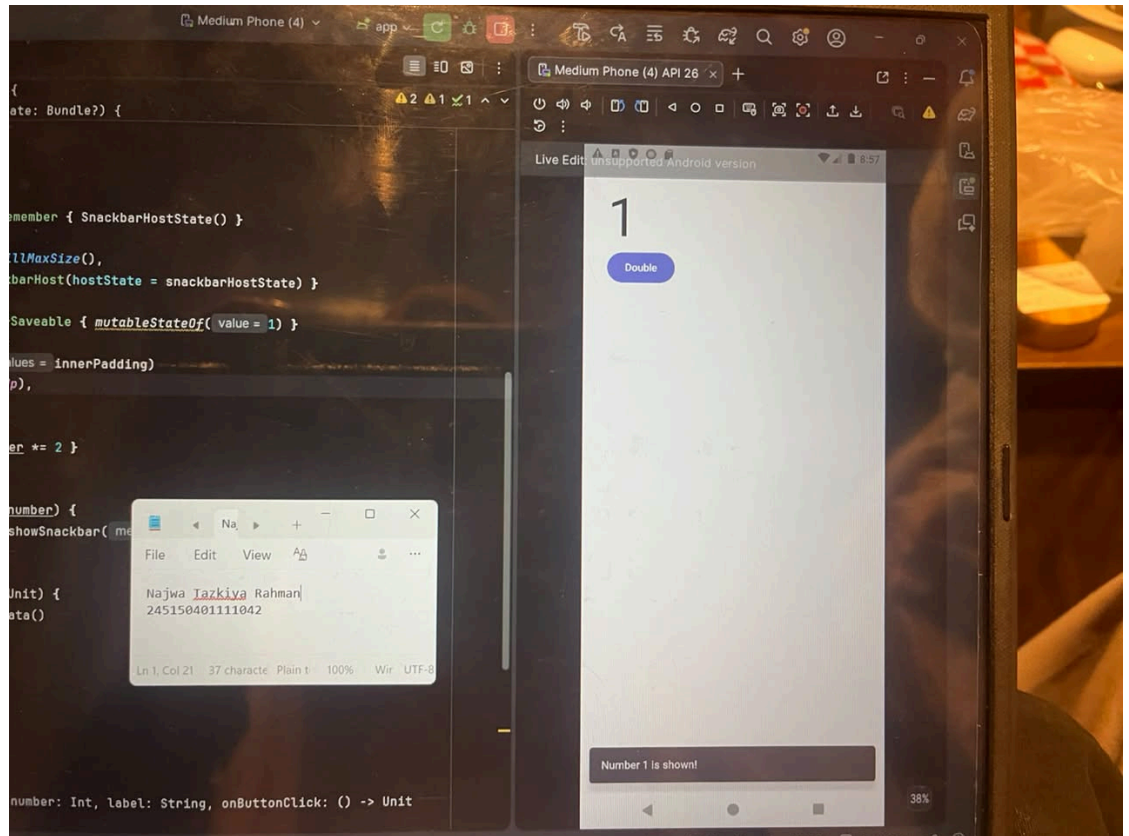


4. Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi onCreate(). Tidak ada perubahan yang dilakukan di dalam fungsi composable CounterScreen().

Aktivitas Percobaan

1. Jalankan program,

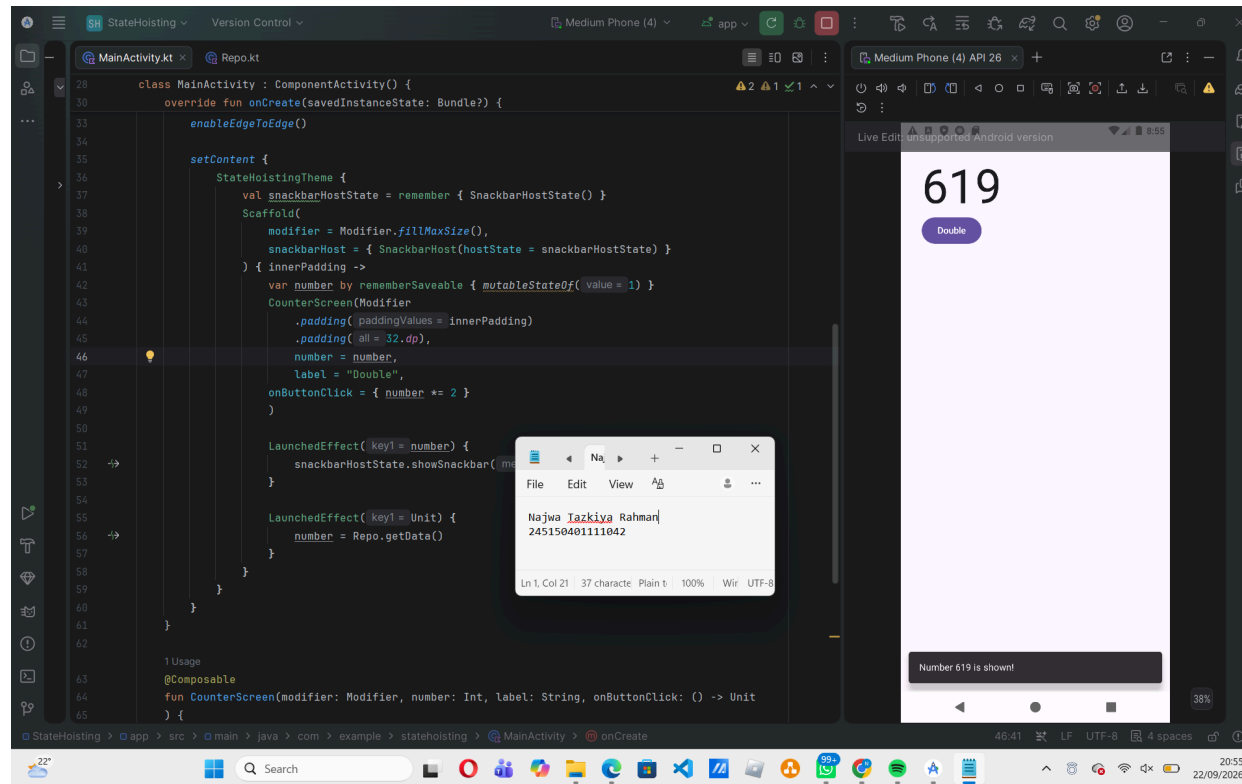
- Pada saat pertama kali program dijalankan, tanpa mengklik button “Double” apa yang ditampilkan di dalam layar?



Pada awal aplikasi dijalankan, nilai awal number akan ditampilkan, yaitu 1. SnackBar juga dapat menampilkan pesan berdasarkan nilai awal tersebut karena terdapat `LaunchedEffect(number)`.

b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah sekitar 2 detik, `Repo.getData()` selesai melakukan simulasi pengambilan data dan menghasilkan angka acak antara 100 sampai 999. Nilai tersebut kemudian diberikan kepada number sehingga angka yang tampil pada layar berubah dari nilai awal menjadi angka hasil `Repo.getData()`. SnackBar juga akan menampilkan nilai baru tersebut karena perubahan number memicu `LaunchedEffect(number)`.



c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

```

LaunchedEffect( key1 = Unit ) {
    number = Repo.getData()
}

```

Perubahan angka pertama kali dipicu oleh Repo.getData() selesai selama sekitar 2 detik, nilai number diganti dengan angka hasil pengambilan data. Kemudian perubahan number memicu

```

LaunchedEffect( key1 = number ) {
    snackbarHostState.showSnackbar( message = "Number $number is shown!" )
}

```

sehingga Snackbar

menampilkan nilai yang baru.

3. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,

a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?

LaunchedEffect(state) digunakan untuk menjalankan side-effect berdasarkan suatu state yang digunakan sebagai pemicu. Jika nilai state tersebut berubah, maka LaunchedEffect akan dijalankan kembali. Dalam percobaan ini, state yang digunakan adalah number, sehingga dituliskan sebagai LaunchedEffect(number). Sementara itu, LaunchedEffect(Unit) digunakan untuk menjalankan side-effect ketika Composable pertama kali dijalankan. Jadi, perbedaan utamanya adalah

LaunchedEffect(state) dapat dijalankan kembali ketika nilai state berubah, sedangkan LaunchedEffect(Unit) digunakan untuk menjalankan proses tanpa bergantung pada perubahan state tertentu.

b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) Dijalankan?

LaunchedEffect(state) dijalankan saat Composable pertama kali masuk ke composition dan dijalankan kembali ketika nilai state yang menjadi key berubah. Sedangkan LaunchedEffect(Unit) dijalankan ketika Composable pertama kali masuk ke composition dan tidak dijalankan kembali hanya karena number (state) berubah.